Can Decision Models Replace Tool Calling for Coding Agents?

Aziz Muminov Alexandra Ilyina

Preprint draft — 27 October 2026

Abstract

Coding agents increasingly carry large tool catalogs, and every request re-sends every tool definition. We test whether a *decision model* (DM), a model that returns a probability for each of a set of caller-supplied options without generating text, can take the catalog's place. We built a sandboxed coding-agent benchmark: an unmodified open-source agent works on 30 Python repositories whose hidden tests require facts available only through a fictional company's internal catalog of 100 or 300 tools. Instead of the catalog, the agent gets one universal tool: it states what it needs, a DM picks the matching tools, and the agent calls one. Across 1,260 runs with two LLMs, the universal tool matched loading every definition (98–99% vs. 97–99% success on tasks that need the catalog) while reading 81% less context per run with 100 tools and 92–93% less with 300, at a quarter to an eighth of the cost. Two conditions matter. The agent must be told which services exist: without a one-line list it sometimes never searches. And tools must be selected when the need arises: choosing them upfront from the task prompt solved 14% of tasks whose needs appear mid-task. The DM does not replace the agent's own calls (on a real coding session it predicted the next tool less often than "always use the shell"); it replaces the catalog.

1 Introduction

A coding agent acts through tools: a shell, file reading and editing, search, and, increasingly, the tools of every service its organization exposes through protocols such as MCP [8]: issue trackers, CI systems, configuration services, documentation, databases. Each tool comes with a definition (a description and a parameter schema) that the agent's LLM reads on every request, whether or not the tool is used. In the agent session in which this paper's experiments were built, 51 always-loaded tool definitions took about 59,000 tokens, 93% of the first request, and were re-sent with each of 435 requests; the agent used 10 of them (§3). The coding agent we test documents the same problem: tool servers "tend to add a lot of tokens and can easily exceed the context limit" [10].

Decision models (DMs), released by several vendors in 2026 [7, 11, 17], are built for choices. A DM reads a state and answers typed questions (yes/no, pick one of these options, give a score) with a probability per option, in one forward pass, with no generated tokens, in well under a second and for about $0.04 per million input tokens. Finding the right tool in a catalog is such a choice. We therefore replace the catalog with a *universal tool* (Figure 1): the agent keeps its core coding tools and gets two more, `find_tool(intent)`, which a DM resolves against the full catalog at the moment the agent asks, and `call_tool(name, arguments)`. The catalog leaves the prompt; the LLM still writes the intent, the arguments and the code.

Whether this works is a question about the whole agent loop, not about single calls: the agent must decide to look for a capability it cannot see, describe it well enough, and use what comes back. We therefore built a coding-agent benchmark for it (§4). An unmodified open-source coding

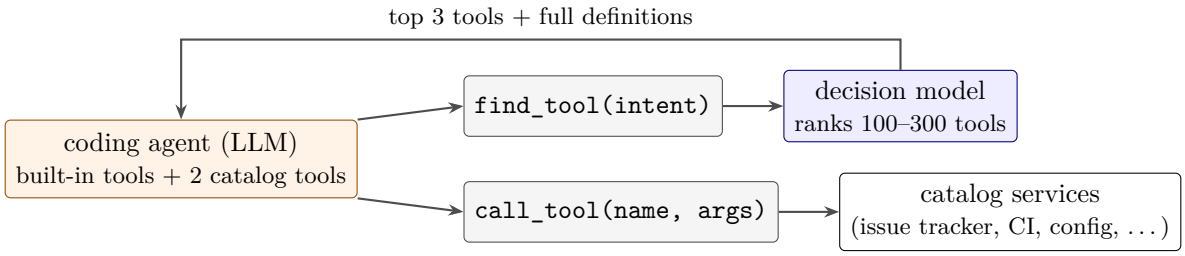


Figure 1: The universal tool. The agent's prompt holds two short tool definitions instead of the catalog; the decision model resolves each stated intent at the moment the agent needs it.

agent [10] runs in an operating-system sandbox on 30 small Python repositories. Their hidden tests require values that only a fictional company's internal services return (a production configuration value, a feature flag, a CI threshold, an API contract), through a catalog of 100 or 300 realistic tools; some needs appear only after a first lookup, and some values do not exist at all. We compare five ways of exposing the catalog, from none to all definitions, with two LLMs, in 1,260 runs. We find:

- **The universal tool matches loading the whole catalog.** With a one-line list of the services the catalog covers, it passed 112 of 114 runs that need the catalog against 113 for native tool calling at 100 tools, and 75 of 76 against 74 at 300 tools (§5, §6).
- **It removes most of the context.** The largest prompt fell from 61–65k to 10–12k tokens at 100 tools and from 169–178k to 10–12k at 300; the context read per run fell by 81% and 92–93%, the cost per run to a quarter and an eighth.
- **Two design choices decide whether it works.** Without the service list, agents sometimes never search for a capability they do not know exists (§7). Choosing tools once from the task prompt instead of at the moment of need solved 14% of tasks whose needs appear mid-task.
- **No condition invented a value**: where the value did not exist, every run abstained.

The DM's role is selection only. A DM cannot write arguments, and on the tool decisions of a real coding session it predicted the agent's next tool less often than always choosing the shell (§8): it replaces the catalog, not the agent's tool calls.

Contributions. (i) A sandboxed coding-agent benchmark for tool catalogs: an unmodified open-source agent, a fictional 100/300-tool catalog, 30 tasks with hidden tests, five tool-exposure conditions, and false-positive and abstention accounting. (ii) Evidence from 1,260 runs that a DM-backed universal tool preserves task success at a fraction of the context, and grows more valuable as the catalog grows. (iii) An analysis of its failure modes and a design recipe. (iv) A measurement of what tool definitions cost in a real coding session, and of why a DM cannot take over the agent's own calls.

2 Background and related work

Decision models. We use OpenRouter's Decisions API [11]: a request carries a *state* (text or JSON) and named questions of type yes/no, `choice` (one of named options, each with a free-text description; returns a distribution) or `score`. We evaluate Microsoft-Decision-1 (MSD1), a post-trained Qwen3.5-9B [7], and TypeSafe Jev 1.14 [17]. In our runs both rejected a `choice` with more than 255 options, and MSD1 requests beyond 32k tokens. The Artificial Analysis Decision Index [1]

and an analysis of it [9] compare DMs with LLMs on standalone tasks and find them about even on tool-call routing; neither places a DM inside an agent.

Too many tools. Retrieving tools before generation is the standard answer to large catalogs [4, 14], and Lee [5] find that selecting a tool is largely retrieval while recognizing that none applies is the hard part. Agent harnesses increasingly defer tool definitions and load them on demand by keyword search; the agent that built this paper did so for some of its tools. We test a learned, semantic version of this idea inside an unmodified coding agent, and measure its effect on end-to-end task success.

Deciding when to call. When2Call [15], SMART [13], "To Call or Not to Call" [18] and Probe&Prefill [16] study whether an LLM should call a tool at all. Coding agents mid-task almost always act, so we do not study this gate here.

Benchmarks. Tool-calling benchmarks such as BFCL [12] score single calls with small menus; agentic benchmarks score task completion [2, 6, 19], and their evaluators are themselves noisy [3]. We therefore built tasks scored by hidden unit tests, in a fictional organization whose tools no model can have seen.

3 Where does tool calling cost context?

We parsed the full transcript of the coding session in which this paper's experiments were built (one agent, 32 user requests, 453 tool calls in 395 decisions, 435 LLM requests). It records the prompt the agent ran with. Its 51 always-loaded tool definitions took about 202,000 characters, about 58,900 tokens (calibrated against the first request, 63,454 tokens before any conversation), and were re-sent with every request: about 25 million of the 162 million context tokens the session read (16%). Two items alone cost about 39,000 tokens per request and were called once between them: a publishing tool (17,400) and 30 cloud-session tools (21,300). The agent used 10 of the 51. Measured per tool, real definitions averaged about 1,150 tokens (MCP tools: about 700), against about 150 in BFCL.

Prompt caching makes these tokens cheap but not free: at the session model's cached-read price ($0.20 per million) they cost about $5 of a $60 session, and they occupy the context window on every request. Without caching (one-off calls, tool lists that change between requests, low traffic) the same definitions are billed at full input price: for 100 tools of 700 tokens, about $280 per thousand requests at $4 per million input tokens, against about $0.50 per thousand DM calls that each select among 200 tools (App. C).

4 A coding-agent benchmark for tool catalogs

The benchmark asks whether an agent that cannot see the catalog still finds and uses the tools it needs, and what each way of exposing the catalog costs in context.

Harness. We run opencode [10] (v1.19.4), an open-source coding agent, unmodified and headless. It keeps its built-in tools (read, edit, glob, grep, list, a to-do list, and a shell restricted to running `pytest`) in every condition; web access is off. The catalog is served by our MCP server over local HTTP.

Table 1: Tool-exposure conditions. Built-in coding tools are present in all of them.

Condition	What the agent sees of the catalog
none	nothing (shows which tasks need the catalog)
native	every catalog tool with its full definition, in every request
filtered	the 5 tools MSD1 ranks highest for the task prompt, chosen once before the run
universal	`find_tool` and `call_tool`; no hint of what the catalog contains
universal + list	the same, plus a one-line list of the services the catalog covers

Sandbox. A first pilot with harness-level permissions only was invalid: the agent read the hidden tests and answer files of its own task through absolute paths (App. A). All reported runs therefore execute as an unprivileged operating-system user with an empty environment, in a fresh copy of the task. The experiment's files and the API key are readable only by root. The LLM is reached through a local proxy that adds the key and admits only the two experiment models, and the catalog server runs as root. We scanned every transcript for file access outside the task folder.

Catalog. A fictional company with 12 internal services (issue tracker, CI, feature flags, configuration, documentation and API-contract registry, warehouse, metrics, deployments, chat, on-call, code search, package registry) exposes 100 tools (Table 7). Each has the shape of a real MCP tool, a multi-paragraph description with parameters, return format, errors, limits and an example call, and averages about 610 tokens, close to the MCP tools measured in §3; the catalog totals about 61,000 tokens. A 300-tool catalog adds 20 services, several deliberately close to an original one (a support desk beside the issue tracker, A/B experiments beside feature flags), for about 184,000 tokens. Every tool answers deterministically from a per-task world file: the right tool with sensible arguments returns the needed fact, anything else an empty list or `NOT_FOUND`.

Tasks. Thirty small Python repositories (Table 8), each scored by hidden tests: *control* (6) bugs fixable from the repository alone; *single* (11) the code must use one value that only a catalog tool returns (a production config value, a flag, a column width, a package version, an SLO target, a service owner, a chat decision, a constant in another repository, a CI threshold); *mid-task* (7) the first lookup (a ticket, a chat thread, a docs page, a CI log) points to a second service, so the second need cannot be predicted from the prompt; *contract* (1) implement a parser from an API contract; and *absent* (5) the value does not exist anywhere the agent can reach (a missing record, or a system not in the catalog). Every value prompt ends with the same escape clause: if the authoritative value cannot be found, leave it unchanged and add a `TODO: value not found` comment. The right outcome on absent tasks is to abstain; changing the value is a fabrication. We validated every task: hidden tests fail on the original code and pass with a reference solution.

Conditions. Table 1. In the two universal conditions the agent sees two tools instead of the catalog (Figure 1): `find_tool(intent)`, for which MSD1 ranks the catalog against the stated intent and returns the top three tools with their full definitions, and `call_tool(name, arguments)`. With 300 tools, `find_tool` shortlists five tools from each half of the catalog and decides among the ten (the 255-option limit). The *service list* variant adds one line per service to `find_tool`'s description (App. A).

Models and runs. Two LLMs drive the agent: DeepSeek V4.1 Flash and GLM-5.3, both with 1M-token context windows. Each of the 30 tasks ran under all five conditions with both models,

Table 2: Main results, 100-tool catalog (30 tasks × 3 repeats per row). Pass rate over all tasks, with a task-clustered 95% CI; largest prompt and context read per run are means over runs.

Model	Condition	Pass rate [95% CI]	Steps	Largest prompt	Context read / run	Cost / run
DeepSeek	none	0.367 [0.20, 0.53]	7.0	8,193	50,537	$0.0049
	native	1.000 [1.00, 1.00]	7.2	64,536	460,805	$0.0266
	filtered	0.789 [0.63, 0.92]	8.0	11,919	89,322	$0.0080
	universal	0.978 [0.94, 1.00]	8.4	11,269	81,047	$0.0066
	universal + list	**1.000** [1.00, 1.00]	8.0	12,297	88,417	$0.0067
GLM-5.3	none	0.367 [0.20, 0.53]	6.7	6,866	42,714	$0.0040
	native	0.989 [0.97, 1.00]	5.4	60,631	324,141	$0.0233
	filtered	0.789 [0.63, 0.92]	6.3	9,873	60,699	$0.0058
	universal	0.878 [0.78, 0.97]	7.5	9,139	61,954	$0.0059
	universal + list	**0.978** [0.93, 1.00]	6.6	9,593	60,028	$0.0055

Table 3: Pass rate by task category, 100 tools, both models (6 runs per task and condition).

Category (tasks)	none	native	filtered	universal	universal + list
control (6)	1.00	1.00	1.00	1.00	1.00
single (11)	0.00	1.00	0.97	0.94	1.00
mid-task (7)	0.00	0.98	**0.14**	0.79	0.95
contract (1)	0.00	1.00	1.00	1.00	1.00
absent (5)	1.00	1.00	1.00	1.00	1.00

three times (900 runs, 100 tools). We report pass rates with 95% confidence intervals that resample whole tasks, exact McNemar tests on runs paired by task, model and repeat, and the tokens and cost opencode records per request (its cost matched the provider's bill within 1%: $8.84 for the 900 runs).

5 Can a decision model replace the tool catalog?

The universal tool with a service list matches native. On tasks that need a catalog tool it passed 112 of 114 runs, against 113 of 114 for native (Figure 2b); paired by task, model and repeat, one run passed only with the universal tool and two only with native ($p = 1$). It read 81% less context per run (Table 2): the largest prompt fell from 61–65k to 10–12k tokens and the context read over a whole run from 324–461k to 60–88k, at about a quarter of native's cost, for about one extra step per run (the search).

Without the list, the universal tool loses some accuracy. It passed 101 of 114 runs; against native the paired difference is significant (0 vs. 12 discordant runs, $p = 0.0005$), almost entirely through GLM (0.878 overall). §7 shows why.

The tasks need the catalog, and the controls do not. Without a catalog no run passed a task that needs one (0 of 114); every control and absent task passed in every condition (Table 3). No control run touched the catalog.

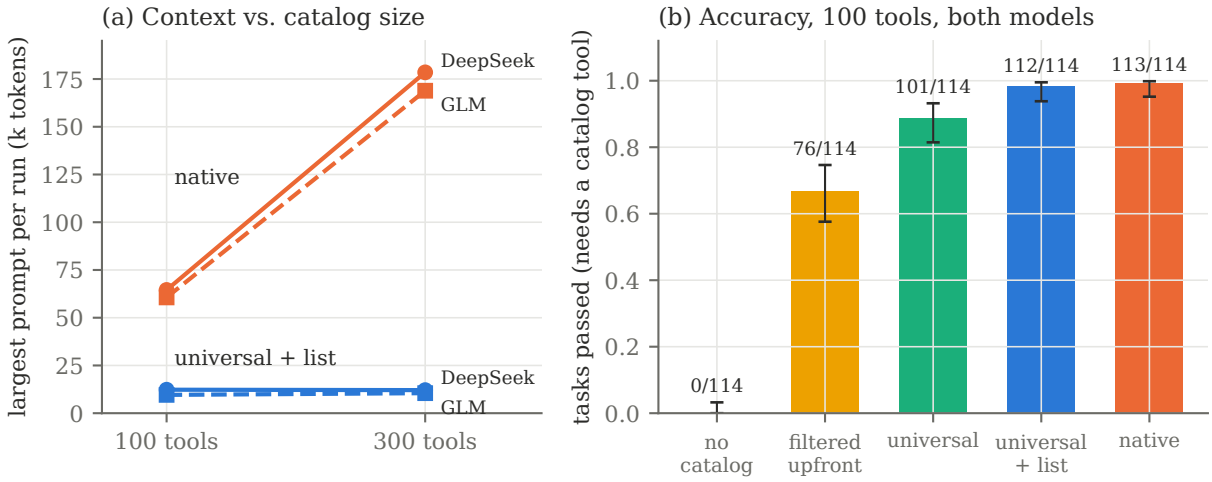


Figure 2: (a) The largest prompt of a run grows with the catalog when every definition is loaded and stays flat with the universal tool (means over runs; 300-tool runs from §6). (b) Share of runs passed on the 19 tasks that need a catalog tool (single, mid-task, contract), 100 tools, both models pooled, with Wilson 95% intervals.

Table 4: 100 vs. 300 catalog tools (repeats 0–1 at both sizes; pass rate over all tasks, largest prompt and context read per run are means over runs).

		Pass rate		Largest prompt		Context read / run	
Model	Condition	100	300	100	300	100	300
DeepSeek	native	1.000	0.967	64.5k	178.5k	459k	1,306k
	universal	0.983	1.000	11.2k	12.0k	80k	90k
	universal + list	1.000	1.000	11.5k	12.1k	79k	87k
GLM-5.3	native	0.983	1.000	60.7k	168.9k	319k	908k
	universal	0.867	0.883	9.3k	9.0k	64k	62k
	universal + list	0.967	0.983	9.6k	10.5k	61k	70k

In short. Yes: with a one-line list of the services that exist, a DM-backed universal tool replaced the catalog without a detectable loss of accuracy and with 81% less context.

6 Does it hold as the catalog grows?

The argument for a universal tool rests on large catalogs, and a larger catalog is also harder for the DM, especially with near-duplicate services.

We repeated native and both universal conditions with the 300-tool catalog, two repeats per task and model (360 runs, 60 per model and condition; billed $9.74). Table 4 compares them with the 100-tool runs of the same repeats.

Native's context nearly triples; the universal tool's barely moves. With 300 tools the largest native prompt reached 169–178k tokens and a run read 0.9–1.3 million; the universal tool with a service list stayed at 10–12k and 70–87k, 92–93% less context read per run (Figure 2a). Native's cost per run rose from $0.023–0.026 to $0.064–0.071 (opencode's estimates; the bill for this batch was 17% higher); the universal tool's from $0.006 to $0.008.

Accuracy held. On the tasks that need the catalog, the universal tool with a service list passed 75 of 76 runs at 300 tools and native 74 of 76 (over all tasks: 119 of 120 and 118 of 120). Native's two failures were DeepSeek runs that never found a mid-task tool among 300 definitions and abstained; the universal tool's one failure was the GLM mid-task run that also failed at 100 tools (§7). None of the accuracy differences in this sweep is significant (universal + list vs. native: 2 vs. 1 discordant runs, $p = 1$; universal without the list vs. native: 2 vs. 7, $p = 0.18$).

The DM was not the bottleneck. The DM's top three contained a needed tool for 86–95% of `find_tool` calls on tasks that need the catalog at 300 tools (87–91% at 100), despite the near-duplicate services; agents that missed searched again. Earlier experiments point the same way (App. C): on BFCL items padded to 200 random tools, MSD1 picked the right tool for 94.7% of items against 86.0% for DeepSeek ($p = 0.004$), and its top five always contained it; on a contamination-free benchmark of near-duplicate fictional tools its top three contained the right tool for every should-call item at 247 tools; and in a pre-registered test on coding-harness tools it chose an acceptable next tool for 98.8% of fresh items against 77.9–88.4% for three LLMs.

In short. The context saving grows with the catalog, and accuracy did not fall within what this sweep can detect.

7 What breaks the universal tool?

An agent does not search for what it does not know exists. Of the 13 failures of the universal tool without a service list (100 tools), 10 are runs in which the agent never called `find_tool`: it did not suspect that a tool for, say, the SLO target or the feature flag existed, and abstained. Eleven of the 13 are GLM runs. In no failure did the DM fail to return a needed tool when asked. The one-line service list removes this failure mode: with it, every run that needed the catalog searched, and the only two failures are runs that found the right tool for the first step of a two-step chain and did not complete the second.

Selecting tools upfront misses needs that appear mid-task. The filtered condition chose five tools once, from the task prompt. That suffices when the prompt names the need (0.97 on single lookups; the shortlist contained every needed tool in all runs) and fails when a need is revealed by the first lookup: on mid-task tasks the shortlist contained every needed tool in 14% of runs, and 36 of filtered's 38 failures trace to a missing tool. A universal tool does not have this problem because the agent asks when the need arises.

False positives were rare. Table 5 lists the failure signals we recorded. No condition invented a value: on the five absent tasks every run abstained, and every value-task run that did not reach the right value abstained rather than changing it. This depends on the escape clause in the prompt, which makes honest abstention easy. State-changing catalog calls (creating issues, setting flags) occurred about once per 100 native and filtered runs, calls to tool names that do not exist about twice per 100 universal-with-list runs, and one universal run passed without calling the needed tool.

Blocked agents probe their environment. In several runs an agent that could not find what it needed wrote test files that dumped its environment, searched the filesystem for the missing value, or inspected the agent's own database, and ran them through the one shell command it was allowed.

Table 5: False positives and extra activity per run, 100 tools, both models (180 runs per condition; lucky passes out of the 114 runs that need a catalog tool).

Condition	Invented values	State-changing calls	Unknown tool names	Failed calls	Extra calls	Lucky passes
native	0	0.01	0.00	0.55	2.03	0
filtered	0	0.01	0.00	2.35	2.21	0
universal	0	0.00	0.00	0.77	1.04	1
universal + list	0	0.00	0.02	0.67	1.82	0

Table 6: Predicting the tool a real coding agent called next (395 decisions). Options: the 38 tools the agent could call, including tools loadable on demand; macro scores weight the 13 it used equally.

Predictor	Accuracy	Macro precision	Macro recall	Macro F1	Top-5
Always Bash	**0.754**	0.058	0.077	0.066	–
Repeat previous tool	0.597	0.108	0.108	0.108	–
MSD1	0.489	0.311	0.374	**0.310**	0.944
Jev	0.597	0.315	0.389	0.267	0.957

The operating-system sandbox contained every attempt (no protected file was read); harness-level permissions alone did not (App. A).

8 Can the decision model make the calls itself?

In the universal tool the DM only selects; the LLM still decides when to search and writes every call. Could the DM go further and make the agent's tool calls itself? We test the choice of the next tool on a real session and on public agent trajectories, and the filling of arguments on a standard benchmark.

Replaying a real session. We replayed the 395 tool decisions of the session of §3. At each decision both DMs saw the user's latest request and the agent's last eight steps (tool calls and results, clipped), about 2,000 tokens against the agent's median of 340,000, and chose among the 38 tools the agent had. Table 6 shows the result. The shell (Bash) served 75% of decisions, so "always Bash" beats both DMs on accuracy; the DMs are 4–5 times better on macro-F1, i.e. on the rare tools, and their top-5 contains the agent's tool 94–96% of the time. The DMs' "errors" are largely equivalent tools: reading a file with the Read tool where the agent used `cat`.

Real agent trajectories. On 300 steps from 60 resolved OpenHands trajectories on SWE-rebench, no system beat "always run bash" (0.530): Jev reached 0.570 (95% CI of the difference −2.0 to +9.7 points, trajectory-clustered), two LLMs 0.527–0.533, and MSD1 0.350, chiefly by predicting `finish` on 62 steps of which 3 were real (App. C).

Arguments. A DM can only choose among values present in the request. Choosing every argument with DMs reaches 0.44–0.52 on BFCL single-turn categories, and even answering every DM question with the gold caps this design at 46–67%: the remaining calls need values produced or normalized ("Fremont" → "Fremont, CA"), or arrays and objects.

In short. No. A coding agent's core tool choice is nearly trivial (one general tool, the shell, serves most of it), and the hard part, writing commands and arguments, is generation. That is why the universal tool leaves both to the LLM and hands the DM only what costs context: knowing every tool that exists.

9 Discussion

What decision models replace. Not the agent's own calls: the core coding tools are few, the shell covers most of them, and the hard part is writing the command, which a DM cannot do. But they can replace the part of tool calling that grows with an organization: knowing every tool that exists. Behind one universal tool, a DM let a coding agent use a 300-tool catalog with the context of a handful of tools and without measurable loss of accuracy.

A recipe. (1) Keep the core coding tools native. (2) Put the catalog behind `find_tool` and `call_tool`, resolved by a DM at the moment of need. (3) Tell the agent, in one line per service, what the catalog covers; this cost about 170 tokens for 12 services and 430 for 32, and removed the main failure mode. (4) Do not preselect tools from the task text alone. (5) Run agents that reach internal services inside an operating-system sandbox.

Why it works. Selection is a choice among described options, which is what a DM is trained for; the LLM keeps what needs generation (the intent, the arguments, the code). The saving scales with the catalog while the DM's cost does not: about $0.0001–0.0004 per search.

10 Limitations

- **Ceiling effect.** Native passed about 99% of runs, so these tasks can show that the universal tool loses no accuracy, not that it gains any. Harder tasks, longer sessions or larger catalogs are needed to see whether bloated context hurts native.
- **Synthetic catalog and tasks.** Tool descriptions are templated and more uniform than real ones; tasks are short (5–8 steps on average). Real, overlapping tool descriptions may make retrieval harder.
- **Two inexpensive LLMs** (DeepSeek V4.1 Flash, GLM-5.3) and one DM (MSD1) in the harness; frontier models were not run in the harness for cost reasons.
- **Small 300-tool sweep** (two repeats, 120 runs per condition), so its accuracy comparisons have low power.
- **Abstention was prompted**: every value task offered an explicit way to abstain.
- **The session analysis** (§8, §3) is one session of one agent.

11 Conclusion

Decision models can replace a coding agent's tool catalog. In a sandboxed coding-agent benchmark, a single universal tool resolved by a decision model at the moment of need kept the agent's context flat as the catalog grew from 100 to 300 tools, cutting it by 81–93% with no detectable loss of accuracy, provided the agent is told which services exist. They do not replace the agent's own tool calls, which need generation; the LLM keeps those.

Reproducibility. The harness (catalog generator, MCP server, LLM proxy, sandbox setup, runner), the 30 tasks with hidden tests and reference solutions, all run records and the analysis scripts will be released. The harness experiments cost about $19 in API fees.

Use of AI assistance. Claude Code, Anthropic's coding agent, helped throughout this work: it wrote the harness, catalog, tasks and analysis scripts, ran the experiments, and drafted the paper, under the authors' direction. The authors chose the research questions, reviewed the code, results and text, and are responsible for the content. The session analysed in §8 and §3 is this assistant's own session.

References

[1] Artificial Analysis. Artificial analysis decision index v1.0. https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026.

[2] Chaithanya Bandi et al. MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers. *arXiv preprint arXiv:2602.00933*, 2026.

[3] Vishvesh Bhat, Jay Vaghasiya, Muhammad Ahmed Mohsin, and Asad Aali. Benchmarking the benchmarks: A validity audit of tool-calling evaluation. *arXiv preprint arXiv:2607.02577*, 2026.

[4] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.

[5] Janghoon Lee. Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions. *arXiv preprint arXiv:2609.18672*, 2026.

[6] Junlong Li et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.

[7] Microsoft. Not every decision needs an LLM: Getting started with Microsoft-Decision-1. https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, October 2026. Microsoft Community Hub, Educator Developer Blog.

[8] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[9] Randal S. Olson. When to use a decision model instead of an LLM. Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[10] opencode contributors. opencode: an open-source coding agent. https://opencode.ai, version 1.19.4, 2026.

[11] OpenRouter. Decisions API reference. https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026.

[12] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 48371–48392, 2025.

[13] Cheng Qian, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. SMART: Self-aware agent for tool overuse mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[14] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

[15] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2Call: When (not) to call tools. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[16] Chung-En Sun, Linbo Liu, Ge Yan, Zimo Wang, and Tsui-Wei Weng. LLM agents already know when to call tools – even without reasoning. *arXiv preprint arXiv:2605.09252*, 2026.

[17] TypeSafe AI. Jev 1.14 decision model. `https://typesafe.ai`; served via OpenRouter as `typesafe/jev-1.14`, 2026.

[18] Qinyuan Wu, Soumi Das, Mahsa Amani, Arijit Nag, Seungeon Lee, Krishna P. Gummadi, Abhilasha Ravichander, and Muhammad Bilal Zafar. To call or not to call: A framework to assess and optimize LLM tool calling. *arXiv preprint arXiv:2605.00737*, 2026.

[19] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Harness and sandbox details

Permissions. opencode ran with `"*": "deny"` and allowed only read, edit, glob, grep, list, the to-do list, the catalog tools, and shell commands matching `python -m pytest*`, `python3 -m pytest*` or `pytest*`; web fetch and search were denied, and so was access outside the project directory.

Why the operating-system sandbox was necessary. A first pilot ran the agent as root with these permissions only. The file tools nevertheless opened absolute paths outside the project: in several runs the agent read the hidden tests, the world files with the answers, and the task builder, and the no-catalog condition "passed" tasks it could not solve. A configuration file the agent could read also contained the API key. We discarded that pilot and moved to the setup of §4: an unprivileged user, an empty environment, root-only experiment files, the key held by a local proxy, and the catalog served over HTTP by a root process. Two further pitfalls: opencode takes its project directory from the `PWD` environment variable rather than the process's working directory, and resolves relative paths against the project; with either wrong it silently loads a different configuration.

Leak audit. 75 of 1,260 runs mentioned a path outside their task folder. All were inspected: pytest's own interpreter path, file searches that matched nothing, and directory names listed by agent-written test files. No protected file was read.

`find_tool` descriptions (verbatim). Universal: "Find a tool in Quillmark's internal tool catalog. Describe in plain words what you need to look up or do (for example: 'get the production value of a configuration key'). Returns the best-matching tools with their descriptions and parameter schemas. Then run one with call_tool." With the service list, followed by "The catalog covers these internal services:" and one line per service (name and the description in Table 7). `call_tool`: "Run a catalog tool found with find_tool. Pass its exact name and an arguments object matching its parameters." The DM sees, per tool, the first line of its description and its parameter names, and the question "Which tool from the catalog best serves what the agent wants to do?".

Table 7: Services in the fictional catalog. The first twelve make up the 100-tool catalog; the 300-tool catalog adds twenty more, ten tools each, several deliberately close to an original service.

Service	What it covers	Tools
Tracklet	the issue tracker (tickets, bugs, sprints)	10
Buildwave	the CI system (pipelines, runs, jobs, logs, test reports)	10
Flagpole	the feature-flag service (flags, rollouts, segments)	9
Confman	the runtime configuration service (keys per environment)	9
Lorebook	the internal documentation wiki and API-contract registry	9
Datavault	read-only access to the analytics warehouse	9
Pulse	metrics, dashboards, alerts and SLOs	9
Shipyard	deployments, releases and service ownership	9
Murmur	team chat (channels, threads, messages)	7
Rota	on-call schedules and shift swaps	5
Grepline	cross-repository code search	7
Cratedepot	the internal package registry	7
Helpdesk	the customer-support ticket system	10
Abacus	A/B experiments and variant assignment	10
Settingshub	per-customer account settings (not platform configuration)	10
Kbase	the customer-facing knowledge base articles	10
Ledgerly	the finance general ledger and journal entries	10
Crmline	the sales CRM (accounts, contacts, deals)	10
Peopleops	the employee directory and org chart	10
Iamctl	identity and access management (roles, grants)	10
Bucketd	object storage buckets and files	10
Queueline	message queues and consumer lag	10
Mailroom	transactional email templates and sends	10
Indexer	search index management (mappings, reindex jobs)	10
Edgecache	CDN cache rules and purges	10
Dnsctl	DNS zones and records	10
Etlflow	batch ETL workflows and their runs	10
Modelreg	the ML model registry and deployments	10
Statusboard	the public status page and incidents	10
Stockkeep	warehouse inventory levels	10
Auditlog	the security audit log	10
Cronsched	scheduled jobs (cron) and their history	10

B Task suite

Prompts are shown without the escape clause that every value task carries: "If you cannot find the authoritative value, do not guess: leave X unchanged and add a comment line containing `TODO: value not found`."

Table 8: The 30 tasks, with the catalog tools each needs (any one tool per group).

Category	Task	Prompt	Needed tools
control	`c_chunk`	chunk in batching/chunk.py drops the last partial chunk. Fix it.	–
control	`c_dedupe`	dedupe in collections_util/dedupe.py loses the original order. Fix it so tests pass.	–
control	`c_duration`	parse_duration in timeutil/duration.py only understands hours. Make tests/test_duration.py pass.	–
control	`c_leap`	is_leap in calendar_util/leap.py is wrong for some years. Fix it.	–
control	`c_paginate`	tests/test_paging.py is failing. Fix paginate in listing/paging.py.	–
control	`c_slug`	The tests in tests/test_slug.py are failing. Fix textkit/slug.py so they pass.	–
single	`s_batch`	The architecture group agreed on a new maximum export batch size in the #arch chat channel. Update BATCH_SIZE in export/limits.py accordingly.	`murmur_search_messages` or `murmur_get_thread`
single	`s_coverage`	Make COVERAGE_MIN in quality/thresholds.py match the coverage threshold enforced by the analytics-ci pipeline.	`buildwave_get_pipeline_config`
single	`s_emaillen`	Set MAX_EMAIL_LEN in accounts/validation.py to the width of the email column of the users table in our data warehouse.	`datavault_describe_table` or `datavault_run_query`
single	`s_owner`	LEDGER_OWNER in ops/ownership.py is outdated. Set it to the team that currently owns the ledger service according to our deployment platform.	`shipyard_get_service_owner`
single	`s_pagesize`	Set RESULTS_PER_PAGE in search/settings.py to the production value of the feature flag that controls it.	`flagpole_get_flag`
single	`s_qmhttp`	Bump QM_HTTP_MIN in deps/pins.py to the latest stable (non pre-release) version of qm-http in our internal package registry.	`cratedepot_list_versions` or `cratedepot_get_package`
single	`s_ratelimit`	Our rate limiter's default is out of date. Update DEFAULT_PER_MINUTE in ratelimiter/limits.py to the current production platform setting.	`confman_get_config_value` or `confman_export_config`

Category	Task	Prompt	Needed tools
single	`s_retention`	Set RETENTION_DAYS in retention/policy.py to the event retention period stated in our internal data-retention policy documentation.	`lorebook_search_docs` or `lorebook_get_page`
single	`s_retry`	Update MAX_ATTEMPTS in payments/settings.py to the current production platform setting.	`confman_get_config_value` or `confman_export_config`
single	`s_slo`	Set P99_BUDGET_MS in checkout/timeouts.py to the p99 latency target of checkout-api's service level objective.	`pulse_list_slos` or `pulse_get_slo`
single	`s_upload`	MAX_UPLOAD_MB in uploads/limits.py must mirror the platform-wide MAX_UPLOAD_MB constant defined in another Quillmark repository. Update it.	`grepline_find_symbol` or `grepline_search_code` or `grepline_get_file_at_ref`
midtask	`m_crypto`	The deps-audit CI pipeline is failing. Find out why and fix the pin in deps/crypto_pin.py.	`buildwave_get_job_log` or `buildwave_get_test_report`; `cratedepot_check_vulnerabilities` or `cratedepot_get_changelog`
midtask	`m_cutoff`	Apply the nightly cutoff change that the data team discussed in the #data chat channel to pipeline/schedule.py.	`murmur_search_messages` or `murmur_get_thread`; `tracklet_get_issue`
midtask	`m_digest`	Implement ticket QM-511.	`tracklet_get_issue`; `flagpole_get_flag`
midtask	`m_discount`	Implement ticket QM-412 in checkout/pricing.py.	`tracklet_get_issue`; `flagpole_get_flag`
midtask	`m_ledger_timeout`	Implement ticket QM-510.	`tracklet_get_issue`; `confman_get_config_value` or `confman_export_config`
midtask	`m_statuses`	Implement ticket QM-513.	`tracklet_get_issue`; `datavault_describe_table` or `datavault_run_query`
midtask	`m_webhook`	Align WEBHOOK_MAX_RETRIES in webhooks/delivery.py with what our internal documentation says about webhook delivery.	`lorebook_search_docs` or `lorebook_get_page`; `lorebook_get_api_contract`
contract	`k_invoice`	Implement parse_invoice in billing_client/invoice.py for the billing service's v3 invoice payload, following the billing team's published API contract.	`lorebook_get_api_contract`
absent	`a_fraud`	Update SCORE_THRESHOLD in fraud/settings.py to the current production platform setting.	–
absent	`a_holidays`	Update PUBLIC_HOLIDAYS in hr/policy.py to the number recorded in our HR system for this year.	–

Category	Task	Prompt	Needed tools
absent	`a_seats`	Set MAX_SEATS in billing/plans.py as specified in ticket QM-999.	–
absent	`a_vendorfee`	Update PAYOUT_FEE_BPS in payouts/settings.py to the fee currently listed in our payment vendor's partner portal.	–
absent	`a_webhook_v4`	Set V4_MAX_RETRIES in webhooks/v4.py to the max_retries value in the API contract webhook-delivery-v4.	–

C Supporting experiments

These experiments, run before the harness, test tool selection by DMs outside an agent loop.

Selection among many tools (BFCL padded). 150 single-call BFCL items, each padded with random distractors from the ≈1,270 distinct BFCL tools to 25, 100 and 200 tools. Right tool chosen, 200 tools: MSD1 0.947, Jev 0.953, DeepSeek 0.860 ($p = 0.004$ vs. MSD1); on an 81-item subset Claude Sonnet 5.5 reached 0.914, with no detectable difference from the DMs (95% CI of the difference −2.5 to +7.4 points). The DMs' top five contained the right tool for every item. The selecting call cost $0.53 per thousand requests for MSD1, $4.58 for DeepSeek and $88.9 for Sonnet, whose requests reached about 63k input tokens.

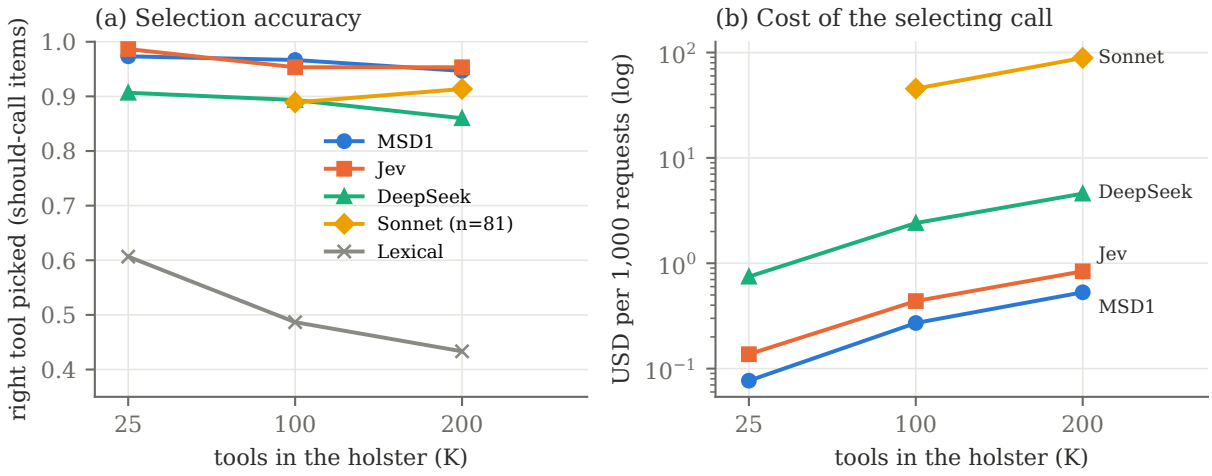


Figure 3: Selection on BFCL items padded with random distractor tools. (a) Right tool chosen (should-call items; $n = 150$ per point; Sonnet on an 81-item subset at $K \geq 100$). (b) Cost of the selecting call (log scale).

Contamination-free selection. On 288 items of a benchmark built on the day of the experiment (fictional companies, tools in families of near-duplicate variants that differ in one implied detail, plus 28 coding-harness tools), with all 247 tools available, MSD1 chose correctly more often than DeepSeek ($p = 0.024$) and MiMo V2.6 Pro ($p = 3 \times 10^{-6}$) and tied GLM-5.3; its top three contained the right tool for every should-call item.

Table 9: Study 3: next-action agreement with resolved OpenHands trajectories (300 steps from 60 trajectories, one per repository). CIs resample whole trajectories. "finish": steps where the system predicted `finish` (3 of the 300 gold steps are `finish`).

System	Agreement	95% CI	Top-2	"finish"	$/1k steps
Always `execute_bash` (majority)	0.530	0.470–0.593	–	0	0
Repeat previous action	0.410	0.360–0.467	–	0	0
Jev	**0.570**	0.510–0.633	0.770	15	0.14
Sonnet 5.5	0.533	0.473–0.597	–	0	19.94
DeepSeek	0.527	0.470–0.587	–	0	0.45
MSD1	0.350	0.293–0.407	0.620	62	0.13

Pre-registered coding-tool selection. Before generating 86 fresh coding-harness items we recorded hypotheses, metric (an acceptable next tool among 247) and test (exact McNemar, Holm over five comparisons); the record is a local timestamped file, not an external registry. MSD1 chose an acceptable tool for 98.8% of items, Jev 90.7%, against 77.9–88.4% for three LLMs; all five hypotheses were supported (Holm $p \leq 0.035$).